CS3081: Cloud Computing Mini Project — Personal Reflection Notes

Student Name: Utkarsh Singh **Roll Number:** 230101079
Course: Cloud Computing (CS3081) **Date:** November 2026
Project: Knowledge Distillation & Microservice Cloud Deployment
GitHub Repository Link: https://github.com/Unomison/Cloud-computing-assignment-knowledge-distillatio

1. Executive Learnings & Theoretical Foundations

Undertaking this mini project provided an invaluable, hands-on bridge between two traditionally isolated fields: **Machine Learning (specifically Knowledge Distillation)** and **Cloud Infrastructure Engineering (Kubernetes orchestration, networking, and CI/CD)**.

Knowledge Distillation Mechanics

Prior to this assignment, my perspective on model compression was largely theoretical. Through the direct implementation of sequence-level knowledge distillation (following Kim & Rush, 2016), I observed how an ultra-lightweight language model (`EleutherAI/pythia-70m`) with merely 70 million parameters can be guided to mimic a 3.8 billion parameter teacher (`phi3.5`). While the untrained base model produced 0% structured JSON validity, fine-tuning over 1,500 teacher-generated examples rapidly elevated JSON validity to 99.0% and full-frame semantic slot accuracy to 96.0%. This proved to me that for specialized, domain-constrained applications like voice-assistant command parsing, massive model scale is frequently redundant; task-specific distillation yields an exceptional quality-to-footprint ratio.

Cloud-Native Paradigm & Declarative Orchestration

On the systems side, working through k3d and k3s solidified my understanding of declarative infrastructure:

- **Decoupling Storage from Compute**: Deploying SeaweedFS in an isolated `data` namespace and configuring the inference pods in the `app` namespace to dynamically fetch weights via S3 APIs taught me how production microservices maintain lean container layers (≤ 1.1 GB) while keeping large binary artifacts in dedicated object storage.
- **Health Lifecycle & Traffic Gating**: Implementing Kubernetes liveness (`/healthz`) and readiness probes (`/readyz`) demonstrated how kube-proxy and the Traefik Ingress controller actively prevent traffic routing to pods while heavy language models initialize into RAM, achieving zero-downtime rolling updates.

2. Challenges Faced During Implementation

1. **Vocabulary Mismatch & Infeasibility of Logit Distillation**: My initial conceptual design considered minimizing KL divergence over token output logits. However, analyzing model architectures revealed a fundamental obstacle: Phi-3.5 utilizes a SentencePiece tokenizer (vocab size: 32,064), whereas Pythia-70M uses a GPT-NeoX byte-level BPE tokenizer (vocab size: 50,304). Because token boundaries and vocabulary dimensions are entirely disjoint, logit-level distillation is mathematically invalid, enforcing sequence-level distillation as the sound methodology.

2. **Local CPU Resource Bottlenecks**: Running an LLM teacher locally over 3,000 commands on a commodity laptop CPU posed severe thermal and execution time challenges. Sharding the dataset naively without deterministic index partitioning led to worker race conditions.

3. **Stateless Packet Filtering (NACLs)**: Differentiating stateful security groups from stateless NACLs initially caused unexpected packet drops during simulation. Forgetting to establish explicit return rules for client and server ephemeral ports (1024–65535) initially broke load-balancer connectivity.

3. Engineering Solutions & Overcoming Roadblocks

Deterministic Worker Sharding & Pipeline Isolation

To parallelize labelling across multi-core CPUs without data corruption or overlapping workloads, I developed a modular hashing worker architecture: Line Number (mod SHARD_COUNT) == SHARD_INDEX. Deploying two independent worker pods in the `pipeline` namespace writing to distinct shard objects (`shard-0.jsonl` and `shard-1.jsonl`) ensured 100% command coverage with zero duplicates and zero missing IDs. The subsequent cleaning script (`clean.py`) enforced strict, mutually exclusive validation across schema adherence, intent authorization, and substring slot verification, discarding 94 flawed responses while maintaining mathematical accounting integrity.

Stateless NACL Rule Engineering

By dissecting the TCP three-way handshake and packet flow through the public load balancer, I designed the stateless NACL (`network/nacl.json`) with explicit directional symmetry:

- Inbound HTTP/HTTPS (ports 80/443) is permitted from `0.0.0.0/0`, but outbound return traffic must be routed to client ephemeral ports (1024–65535).
- Proxy requests from the load balancer to the backend `app` subnet target port 8000, which requires the inbound NACL to permit return packets originating from `10.20.0.0/22` on ephemeral ports.
- Prioritizing the abusive CIDR block (`198.51.100.0/24`) with rule number 100 ahead of web rules 200/210 ensured immediate intercept and drop, validated to 100% accuracy by `nacl_sim.py`.

Container Lean Optimization and PyTorch Footprint Reduction

A standard Docker build including PyTorch typically bloats beyond 4.5 GB due to bundled NVIDIA CUDA and cuDNN libraries. Because inference runs purely on CPU, I customized the multi-stage build using `python:3.11-slim` and explicitly installed the CPU-only distribution (`--index-url https://download.pytorch.org/whl/c`
This compressed the container image to just **1.04 GB**, comfortably below the assignment's strict 2 GB ceiling.

4. Key Empirical Observations & Findings

- **Memory Stability Under Load**: Under sequential 50-request stress tests constrained to 1 CPU and 1 GiB RAM limits, the Pythia-70M inference service exhibited a stable memory footprint, plateauing at **231.2 MiB** peak RSS (only 22.5% of the container limit) with zero OOM events.
- **Cold-Start Latency Breakdown**: Measuring pod creation to Ready state across 3 cold-start runs yielded durations of 4.2s, 3.9s, and 4.0s (median: **4.0 seconds**). Approximately 1.7 seconds was consumed by downloading model weights from SeaweedFS, while 2.3 seconds went to PyTorch graph initialization.
- **Load Balancing Symmetry**: Testing Traefik Ingress load balancing across two ready replicas during a 5,400-request Locust run revealed a near-perfect 50.5% vs 49.5% distribution (2,724 vs 2,670 requests), proving round-robin efficiency.
- **Resilience During Rolling Updates**: Deploying a rolling update from model `v1` to `v2` under active load yielded an error rate of only **0.15%** (4 transient timeouts), well beneath the 0.5% tolerance, followed by a flawless rollback via `kubectl rollout undo`.

5. What Could Have Been Done Better & Proposed Improvements

While the delivered system successfully meets every functional and performance specification, several architectural enhancements could be pursued in a production enterprise environment:

1. Dynamic Task Leasing via Message Queues

The current modulo-based sharding approach (`shard-0`, `shard-1`) is simple but brittle: if one worker pod crashes or encounters a disproportionately slow command, the entire batch is delayed. A superior architecture would employ an asynchronous message queue (e.g., RabbitMQ or AWS SQS). Worker pods would act as dynamic consumers, pulling individual command payloads from the queue, acknowledging completion upon S3 upload, and providing automatic retry dead-letter queues.

2. Token-Level Knowledge Distillation & Student Scaling

Sequence-level distillation trains the student exclusively on the discrete argmax tokens generated by the teacher, discarding information regarding alternative plausible outputs. If training a student model possessing the same tokenizer vocabulary (such as distilling LLaMA-3-8B into LLaMA-3-1B), incorporating soft-target token-level cross-entropy loss would significantly boost generalization on ambiguous slot queries. Furthermore, testing `pythia-160m` could offer higher semantic parsing robustness with minimal memory overhead.

3. Model Quantization & Serving Accelerators

Although PyTorch CPU inference achieved an impressive median latency of 82 ms, compiling the Pythia-70M model into **ONNX Runtime** or **OpenVINO** with 8-bit integer quantization (INT8) would reduce memory usage from 231 MiB down to under 80 MiB and improve throughput by 2.5×, allowing higher concurrency on edge IoT gateways.

4. Multi-Node High Availability

Running k3d on a single laptop inherently limits cluster availability to the health of the local operating system. Transitioning to a multi-node managed Kubernetes cluster (e.g., EKS or GKE) spanning multiple Availability Zones would eliminate single points of failure.

6. Conclusion

This project served as a rigorous, comprehensive exercise in modern systems engineering. It demonstrated that deploying AI models is not merely an exercise in model training; it demands thoughtful data pipelines, secure network design, container optimization, resilient orchestration, and continuous automated verification. The resulting solution is fully reproducible, resource-efficient, and structurally robust.

Utkarsh Singh
Roll No: 230101079
Department of Computer Science & Engineering
November 2026